מספר זהות:

סמסטר א' תשפ"ו
מועד: ב' 24/02/2026 9:00
משך המבחן: שעתיים ועשרים דקות
חומר עזר: כל חומר עזר כתוב מותר

בחינה בקורס: **תכנות מכוון עצמים ושפת ++C**

מרצים: **אמיר קירש, ד"ר איריס גבר**

קטעי הקוד זהים לכל הטורים במבחן
בתום המבחן חובה להחזיר את קטעי הקוד ביחד עם שאלון המבחן

הנחיות כלליות לבחינה:

- המבחן מורכב מחלק אחד ובו:
  - **10 שאלות אמריקאיות. משקל כל שאלה 10 נקודות, סה"כ: 100 נק'.**
  - **4 שאלות אמריקאיות בונוס. משקל כל שאלה 4 נקודות, סה"כ: 16 נק'.**
  - **לא ניתן לצבור יותר מ-100 נקודות במבחן.**
- יש לסמן תשובה אחת לכל שאלה בטופס התשובות. במידה ומספר תשובות נראות נכונות יש לסמן את התשובה הנכונה ביותר.
- בסיום המבחן יש לרשום מס' ת.ז. על גבי טופס המבחן ולוודא שטופס התשובות מוגש יחד עם טופס המבחן.
- המבחן הינו עם חומר פתוח. כל חומר עזר כתוב מותר למעט מכשירים אלקטרוניים למיניהם. אין להעביר חומר עזר בין תלמידים במהלך המבחן.

בהצלחה !

קטע קוד A

```
 1. template<size_t SIZE> class BooleanArray {
 2.   static constexpr size_t BITS = 8;
 3.   static constexpr size_t array_size = (SIZE - 1)/BITS + 1;
 4.   char bool_bits[array_size] {};
 5.   class BooleanBit {
 6.     char* bool_bits;
 7.     size_t byte_index;
 8.     char bit_index;
 9.     size_t index()const {return byte_index*BITS + bit_index;}
10.   public:
11.     BooleanBit(char* bool_bits, size_t byte_index, char bit_index)
12.       : bool_bits(bool_bits),
13.         byte_index(byte_index),
14.         bit_index(bit_index) {}
15.     operator bool() const {
16.       return (bool_bits[byte_index] & (1 << bit_index)) != 0;
17.     }
18.     bool operator=(const BooleanBit& b) {
19.       *this = static_cast<bool>(b); return *this;
20.     }
21.     bool operator=(bool val) {
22.       if(val != *this) {
23.         // cout << "changing index " << index()
24.         //      << " to " << (val?"true":"false") << endl;
25.         if(val) {
26.           bool_bits[byte_index] |= (1 << bit_index);
27.         } else {
28.           bool_bits[byte_index] &= ~(1 << bit_index);
29.         }
30.       }
31.       return val;
32.     }
33.   };
34. public:
35.   static constexpr size_t size = SIZE;
36.   BooleanArray() {}
37.   template<size_t OTHER_SIZE>
38.   BooleanArray(const BooleanArray<OTHER_SIZE>& other) {
39.     for(size_t i = 0; i < SIZE && i < OTHER_SIZE; ++i) {
40.       (*this)[i] = other[i];
41.     }
42.   }
43.   bool operator[] (size_t index) const {
44.     return const_cast<BooleanArray&>(*this)[index];
45.   }
46.   BooleanBit operator[] (size_t i) {
47.     return BooleanBit(bool_bits, i / BITS, i % BITS);
48.   }
```

```
49.   friend ostream& operator<<(ostream& out,
50.                              const BooleanArray<SIZE>& array) {
51.     for(size_t i=0; i<SIZE; ++i) {
52.       if(array[i]) out << i << ' ';
53.     }
54.     return out << endl;
55.   }
56. };
57.
58. int main() {
59.   BooleanArray<5> boolArr;
60.   boolArr[3] = true;
61.   boolArr[4] = false;
62.   cout << boolArr;
63.
64.   // BooleanArray<3> boolArr1a = boolArr;
65.   // cout << boolArr1a;
66.
67.   // BooleanArray<6> boolArr1b = boolArr1a;
68.   // cout << boolArr1b;
69.
70.   // BooleanArray<1> boolArr1;
71.   // boolArr1[0] = true;
72.   // boolArr1[3] = true;
73.   // boolArr1[5] = true;
74.   // boolArr1[6] = true;
75.   // cout << boolArr1;
76.
77.   // BooleanArray<3> boolArr3;
78.   // boolArr3[1] = boolArr3[0];
79.   // boolArr3[0] = true;
80.   // boolArr3[2] = boolArr[0];
81.   // cout << boolArr3;
82. }
```

קטע קוד B

```
 1. // strings, counted in a map
 2. using Groups = map<std::string, size_t>;
 3.
 4. class MyString {
 5.  static Groups groups;
 6.  static Groups::iterator joinGroup(const string& s) {
 7.    auto itr = groups.find(s);
 8.    if(itr == groups.end()) {
 9.      // see explanation regarding map::insert at line 41
10.       itr = groups.insert({s, 0}).first;
11.     }
12.     ++(itr->second);
13.     return itr;
14.   }
15.   static void leaveGroup(Groups::iterator itr) {
16.     if(--(itr->second) == 0) groups.erase(itr);
17.   }
18.   // this is how we actually hold our string:
19.   Groups::iterator s_itr;
20. public:
21.   MyString(const string& str): s_itr(joinGroup(str)) {}
22.   MyString(const char* str = ""): s_itr(joinGroup(str)) {}
23.   ~MyString() { leaveGroup(s_itr); }
24.   MyString(const MyString& s): s_itr(s.s_itr) { s_itr->second++; }
25.   MyString& operator=(MyString s) {
26.     leaveGroup(s_itr);
27.     s_itr = s.s_itr;
28.     s_itr->second++;
29.     return *this;
30.   }
31.   MyString operator+(const MyString& s) const {
32.     return s_itr->first + static_cast<const char*>(s);
33.   }
34.   operator const char*() const { return s_itr->first.c_str(); }
35.   size_t count() const { return s_itr->second; }
36. };
37.
38. // in MyString.cpp
39. Groups MyString::groups;
40.
41. // explanation regarding map::insert (line 10):
42. // map.insert returns std::pair<iterator, bool>
43. // 'first' is an iterator pointing to the new item or to
44. // an item with the same key that already existed, which
45. // prevented the insertion
46. // 'second' is whether insertion was performed
47.
```

```
48. int main() {
49.   std::array<MyString, 4> sarr;
50.   sarr[0] = "hi";
51.   sarr[1] = sarr[0];
52.   sarr[2] = sarr[1] + "";
53.   auto is_single = [](const auto& s) { return s.count() == 1; };
54.   cout << count_if(sarr.begin(), sarr.end(), is_single) << endl;
55.   sarr[2] = "bye";
56.   cout << count_if(sarr.begin(), sarr.end(), is_single) << endl;
57.   // sarr[2] = sarr[2];
58.   // cout << MyString("MyString").count() << endl;
59.   cout << sarr[2] << endl;
60. }
```

קטע קוד C – לשאלות הבונוס

```
class Length {
public:
  enum class measure { km, m, cm, mm };
  Length(long double _val, measure _m, string _label = "")
  : val(_val), m(_m), label(std::move(_label)) {
    cout << "Length with label: " << *this << endl;
  }
  friend ostream& operator<<(ostream& out, const Length& l) {
    out << l.val << measure_desc[(int)l.m];
    if (!l.label.empty())
      out << " (" << l.label << ")";
    return out;
  }
  long double toMeasure(measure convertTo) const {
    if (convertTo != m)
      return val / conv[(int)m] * conv[(int)convertTo];
    else return val;
  }
  Length operator+(const Length& other) const {
    string the_label;
    if(label.empty()) {
      the_label = other.label;
    }
    else if(other.label.empty() || label == other.label) {
      the_label = label;
    }
    else {
      throw std::invalid_argument("different labels");
    }
    measure convertTo = (m < other.m) ? m : other.m;
    return Length( toMeasure(convertTo) +
                   other.toMeasure(convertTo),
                   convertTo, std::move(the_label) );
  }
  Length& operator()(string a_label) {
    label = std::move(a_label);
    return *this;
  }
private:
  long double val;
  measure m;
  string label;
  static char measure_desc[4][5];
  static unsigned int conv[4];
};

 char Length::measure_desc[][5] = { "km", "m", "cm", "mm" };
 unsigned int Length::conv[] = { 1, 1000, 100000, 1000000 };
```

```
Length operator "" _km(long double d) {
  return Length(d, Length::measure::km);
}
Length operator "" _m(long double d) {
  return Length(d, Length::measure::m);
}
Length operator "" _cm(long double d) {
  return Length(d, Length::measure::cm);
}
Length operator "" _mm(long double d) {
  return Length(d, Length::measure::mm);
}

int main(){
  auto dist1 = 25.0_km ("distance");
  vector dists = {
    10.0_m ("width"),
    20.0_m ("distance"),
    30.0_m
  };
  for(const auto& dist : dists) {
    try {
      cout << dist1 + dist << endl;
    }
    catch(...) {
      cout << "exception" << endl;
    }
  }
}
```

בהצלחה !